

DarshanEase Technical Documentation

Project darshanEase — Smart Temple Darshan Ticket Booking Platform	Status Draft (technical reference)
Core stack MVC backend • JWT + bcrypt • Mongoose models (with JSON-file fallback) • React Router DOM v6	Primary entities Users • Temples • DarshanSlots • Bookings • Donations

1. Project Overview and Purpose

darshanEase is a Smart Temple Darshan Ticket Booking platform designed to help devotees discover temples, view available darshan slots, place bookings, and receive a ticket that can be verified at entry. The platform also supports donations as a first-class domain concept, linked back to users and temples for traceability.

The system is documented as an MVC-style web application where the backend exposes HTTP APIs and the frontend (React) consumes them. Security is based on **bcrypt** for password hashing and **JWT** for access tokens with a refresh-token mechanism for session continuity. Data is modeled using **Mongoose**, with a **JSON-file fallback** intended for lightweight environments or development scenarios where a database is unavailable.

Scope note — This document describes the platform’s intended technical structure and interfaces at a professional reference level, focusing on architecture, security flow, domain relationships, and booking lifecycle without assuming deployment-specific details.

2. System Architecture

The platform follows an **MVC architecture** to keep responsibilities separated and maintainable:

- **Model** — Mongoose schemas/models for Users, Temples, DarshanSlots, Bookings, Donations; optional JSON persistence fallback.
- **Controller** — request handlers that validate input, coordinate domain actions (e.g., create booking), and shape HTTP responses.
- **View** — React UI (routes, pages, components) consuming APIs; server does not render HTML views.

MVC Request Flow (HTTP)

[Browser / React UI]

| 1) HTTP request (JSON) via fetch/axios

v

[Router / Routes] -> [Middleware] -> [Controller] -> [Service/Domain Logic]

| | | |

| | | v

| | | [Model (Mongoose)]

| | | |

| | | (DB) or (JSON fallback)

v v v v

6) JSON response 2) auth/validation 3) orchestration 4) persistence

^

| 5) Controller formats response + status code

Architecture boundaries

- **Frontend** — React SPA with React Router DOM v6; handles navigation, forms, and displaying booking and ticket state.
- **Backend API** — MVC application exposing REST-style routes for auth, temples, slots, bookings, donations, and ticket verification.
- **Persistence** — primary storage via Mongoose models; optional JSON-file fallback to preserve core workflows when DB access is not available.

3. Security and Authentication Model

Authentication is built around two complementary mechanisms:

- **bcrypt password hashing** — user passwords are stored as salted hashes; plaintext passwords are never persisted.
- **JWT-based sessions** — short-lived access tokens authorize API calls; refresh tokens are used to obtain new access tokens without re-entering credentials.

JWT usage (high level)

- **Access token** — sent on protected API calls (commonly via Authorization header); validated by middleware before hitting controllers.
- **Refresh token** — used only to renew access; should be stored and transmitted more cautiously than access tokens.

JWT + Refresh Token Flow

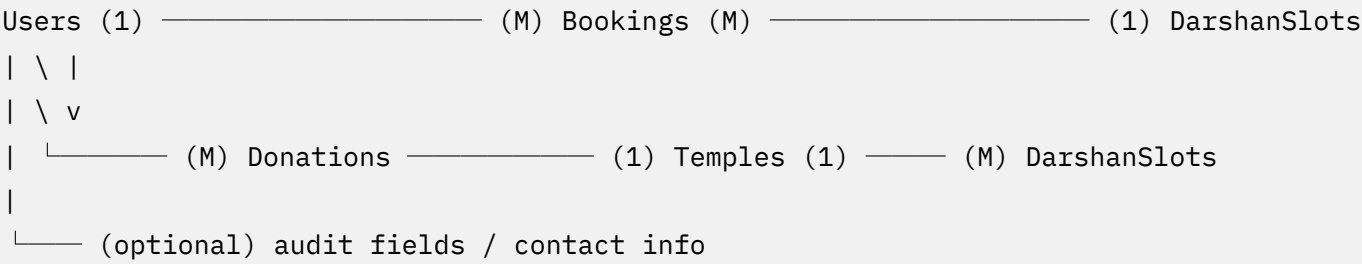
- 1) User logs in with email/phone + password
-> server verifies bcrypt hash
-> returns { accessToken, refreshToken }
- 2) Client calls protected routes with accessToken
-> Auth middleware verifies JWT signature + expiry
-> Controller executes request
- 3) If accessToken expires
Client calls /auth/refresh with refreshToken
-> server validates refreshToken
-> returns new accessToken (and optionally a rotated refreshToken)
- 4) Logout / revoke (recommended)
-> invalidate refresh token server-side where applicable

Security baseline — Protect booking creation, donation creation, and ticket verification endpoints with JWT middleware. Validate all request inputs in controllers before invoking domain logic, and never trust client-provided prices, statuses, or slot availability signals.

4. Database Schema Design

Data is modeled using **Mongoose** with relationships expressed via identifiers/references between collections. A **JSON-file fallback** can mirror these entities for environments without MongoDB, but should preserve the same identifiers and relationship structure to avoid behavior drift across storage modes.

Entity Relationship Diagram (ERD)



Core models (conceptual)

Model	Key fields (minimum)	Relationships
Users	<code>_id</code> , name, contact (email/phone), passwordHash (bcrypt), role (optional), createdAt/updatedAt	1 user → many bookings; 1 user → many donations
Temples	<code>_id</code> , name, location/metadata (as available), createdAt/updatedAt	1 temple → many darshan slots; 1 temple → many donations
DarshanSlots	<code>_id</code> , templeId, startTime/endTime (or time window), capacity, availability/status	Many slots belong to 1 temple; 1 slot → many bookings
Bookings	<code>_id</code> , userId, slotId, bookingStatus, quantity (if supported), ticketCode/qrPayload (concept), createdAt	Many bookings belong to 1 user and 1 slot
Donations	<code>_id</code> , userId, templeId, amount, donationStatus, createdAt	Many donations belong to 1 user and 1 temple

Consistency requirement — Whether stored in MongoDB or a JSON-file fallback, enforce the same relationship rules: bookings must reference an existing user and slot; slots must reference an existing temple; donations must reference an existing user and temple.

5. API Route Reference

Routes are documented as REST-style endpoints aligned to MVC controllers. Protected routes require a valid access token and are authorized by JWT middleware before controller execution.

Area	Method	Path	Purpose	Auth
Auth	POST	/auth/register	Create user; store bcrypt hash	No
Auth	POST	/auth/login	Issue access + refresh tokens	No
Auth	POST	/auth/refresh	Renew access token using refresh token	No*
Temples	GET	/temples	List/search temples	No
Temples	GET	/temples/:temp leId	Get temple details	No
Slots	GET	/temples/:temp leId/slots	List darshan slots for a temple	No
Bookings	POST	/bookings	Create booking for a slot	Yes
Bookings	GET	/bookings/:boo kingId	Fetch booking + ticket details	Yes
Bookings	GET	/me/bookings	List current user's bookings	Yes
Ticket	POST	/tickets/verify	Verify ticket code/booking for entry	Yes
Donations	POST	/donations	Create donation linked to user + temple	Yes
Donations	GET	/me/donations	List current user's donations	Yes

Note — The route list is a clean reference aligned to the documented domain entities and flows. Exact payload shapes and status codes should be defined per controller once validation rules and UI requirements are finalized.

6. Frontend Routing Structure

The frontend is a React single-page application using **React Router DOM v6**. Routes are organized to reflect core journeys: browse temples → choose slot → book → view ticket; plus authentication and user history.

Route Hierarchy (React Router v6)

```
/
├─ /login
├─ /register
├─ /temples
│   └─ /temples/:templateId
│   └─ /temples/:templateId/slots (or section within temple page)
├─ /bookings
│   └─ /bookings/:bookingId (ticket details)
├─ /me/bookings
└─ /me/donations
```

7. End-to-End Booking Workflow

The booking lifecycle is designed to ensure availability and produce a verifiable ticket artifact tied to a booking record.

1. **Discover** — user browses temples and opens a temple detail page.
2. **Select slot** — user views available DarshanSlots for the selected temple and chooses a time window/slot.
3. **Authenticate** — if not logged in, user signs in; access token is used for booking creation.
4. **Create booking** — client calls the Bookings API; controller validates user + slot, checks availability/capacity rules, and persists booking.
5. **Issue ticket** — booking response includes a ticket reference (e.g., ticket code / QR payload) that can be shown in-app.
6. **Verify at entry** — staff/device verifies ticket and updates booking state as appropriate (e.g., “used/checked-in”).

Booking → Ticket Verification Flow

```
[User selects slot]
|
v
POST /bookings (JWT)
| validate user + slot + availability
v
[Booking created] -> ticketCode/qrPayload returned
|
v
[User presents ticket at temple]
|
v
POST /tickets/verify (JWT for staff/admin context)
| lookup booking by ticket reference
| confirm status is eligible + not already used
v
[Verification result] -> allow entry + optionally mark booking as used
```

8. Key Components and Responsibilities

Layer / Component	Responsibilities
Routes (backend)	Map HTTP paths to controllers; apply auth middleware for protected resources.
Auth middleware	Verify JWT, attach user context to request, reject unauthorized requests with precise HTTP status codes.
Controllers	Validate inputs, enforce business rules, coordinate models/services, return consistent responses.
Mongoose Models	Define schemas, relationships, and persistence behavior for Users, Temples, DarshanSlots, Bookings, Donations.
JSON-file fallback	Alternative persistence mechanism maintaining the same entity structure; useful for development or offline constraints.
React Router (frontend)	Route hierarchy for authentication, browsing, booking creation, and ticket display/history.
Ticket verification	Endpoint/controller path that validates ticket reference against Booking state and enforces “single use” semantics.

9. Implementation Notes and Conclusion

- **Keep MVC strict** — avoid pushing persistence logic into controllers; keep validation and orchestration clear and testable.
- **JWT lifecycle** — treat refresh-token handling as a security-critical feature; ensure token renewal is explicit and auditable where possible.
- **Model integrity** — enforce referential integrity at the application layer (especially important for JSON-file fallback).
- **Route consistency** — standardize error shapes and status codes across controllers to simplify frontend handling and reduce edge-case UI logic.

darshanEase combines an MVC backend, JWT + bcrypt security, Mongoose-backed data modeling with a JSON persistence fallback, and a React Router DOM v6 frontend to support a

clear end-to-end darshan booking and ticket verification experience. This document serves as the baseline reference for implementation and review, and can be extended with concrete payload schemas, validation rules, and deployment details as the project matures.